

Programação Orientada a Objetos Aula 10

João Choma Neto
joao.choma@gmail.com

Chamada está em

- <https://github.com/JoaoChoma/POO>

Polimorfismo

- Polimorfismo é um dos conceitos fundamentais da Programação Orientada a Objetos (POO) que permite que um objeto de uma determinada classe seja tratado como se fosse um objeto de sua classe pai ou de uma interface que ele implemente
- Isso significa que um mesmo **método** pode ser utilizado por diversas **classes**, porém cada uma delas pode implementá-lo de forma diferente, de acordo com suas necessidades específicas

Polimorfismo

- O polimorfismo pode ser implementado de duas maneiras
 - polimorfismo de subtipo
 - polimorfismo paramétrico

Polimorfismo de subtipo

- O polimorfismo de subtipo é aquele que permite que um objeto de uma classe filha seja tratado **como** um objeto de sua classe pai

Polimorfismo de subtipo

- Polimorfismo de **subtipo** é quando uma classe filha pode ser tratada como uma classe mãe, ou seja, um objeto de uma classe filha pode ser referenciado como um objeto de sua classe mãe
- Um exemplo de polimorfismo de subtipo é uma classe mãe Animal e suas classes filhas Cachorro e Gato. A classe mãe possui um método emitirSom(), que é sobrescrito pelas classes filhas

```
public class Animal {  
    public void emitirSom() {  
        System.out.println("Emitindo som genérico.");  
    }  
}
```

```
public class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Au Au!");  
    }  
}
```

```
public class Gato extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Miau!");  
    }  
}
```

Polimorfismo de subtipo

- Pode-se criar um array de animais que contém tanto objetos da classe **Cachorro** quanto da classe **Gato**
- Ao percorrer o array e chamar o método emitirSom() de cada objeto, o método correspondente da classe filha será executado

Polimorfismo de subtipo

- `Animal[] animais = new Animal[2];`
- `animais[0] = new Cachorro();`
- `animais[1] = new Gato();`

- `for (Animal animal : animais) {`
- `animal.emitirSom();`
- `}`

Polimorfismo de subtipo

- A saída esperada será:
 - Au Au!
 - Miau!

Polimorfismo de subtipo

- Observe que o mesmo método `emitirSom()` da classe mãe foi chamado para ambos os objetos do array
- ATENÇÃO o método DA CLASSE FILHA correspondente foi executado devido ao polimorfismo de subtipo

Polimorfismo paramétrico

- O polimorfismo paramétrico ocorre quando uma **classe** é capaz de trabalhar com diferentes **tipos** de dados de maneira genérica
- O polimorfismo paramétrico, também conhecido como **sobrecarga de método**, ocorre quando vários métodos possuem o mesmo nome, mas diferentes assinaturas, permitindo que cada um deles seja utilizado de acordo com o tipo dos parâmetros passados

Polimorfismo paramétrico

- Em outras palavras, uma classe ou método pode ser parametrizado com um tipo de dados genérico, permitindo que ele possa ser reutilizado com diferentes tipos de dados específicos

Polimorfismo paramétrico – Exemplo

- Classe `ArrayList` da biblioteca padrão do Java
- Essa classe é uma coleção de elementos que podem ser de qualquer tipo, permitindo que os dados armazenados sejam facilmente manipulados sem a necessidade de criar uma classe específica para cada tipo

Polimorfismo paramétrico – Exemplo

- Podemos criar
 - ArrayList para armazenar uma lista de números inteiros
 - ArrayList para armazenar uma lista de Strings
- Criamos dois objetos ArrayList
 - Um para armazenar uma lista de números inteiros
 - Um para armazenar uma lista de strings
- O tipo de dados utilizado é especificado dentro dos colchetes < > - <Integer> e <String>

Polimorfismo paramétrico – Exemplo

- `ArrayList<Integer> numeros = new ArrayList<Integer>();`
- `numeros.add(1);`
- `numeros.add(2);`
- `numeros.add(3);`

- `ArrayList<String> nomes = new ArrayList<String>();`
- `nomes.add("João");`
- `nomes.add("Maria");`
- `nomes.add("Pedro");`

Polimorfismo paramétrico – Exemplo

- O método **add()** da classe **ArrayList** aceita os tipos de dados específicos definidos para cada objeto
- Dessa forma, podemos adicionar elementos do tipo **Integer** em **numeros** e elementos do tipo **String** em **nomes**
- Essa flexibilidade é uma característica fundamental do polimorfismo paramétrico

Atividade polimorfismo

- Entrega: <https://forms.gle/UbRwCBzzDCRQm8ht7>

Links

- <https://netbeans.apache.org/download/nb17/index.html>
- <https://www.oracle.com/br/java/technologies/downloads/#jdk19-windows>
- <https://github.com/JoaoChoma/POO>

Referências utilizadas

- ASCENCIO, A. F. G. Fundamentos da programação de computadores. 2ª. Edição. São Paulo: Pearson Prentice Hall.
- <https://www.oracle.com/>

Programação Orientada a Objetos Aula 10

João Choma Neto
joao.choma@gmail.com